Taller: Diseño y Construcción de una Criptomoneda

Fundamentos de Blockchain, Criptografía y Sistemas Descentralizados

Elaborado por: **Javier Ochoa**
Docente – Ingeniería de Software
Universidad Manuela Beltrán (UMB)

Resumen

Este taller guía a los estudiantes en la construcción, desde cero, de una criptomoneda educativa funcional sobre una blockchain propia. A lo largo de tres semanas se abordan los fundamentos matemáticos y criptográficos que sostienen sistemas como Bitcoin o Ethereum: funciones hash, estructuras de datos encadenadas, criptografía asimétrica y firmas digitales, mecanismos de consenso, redes de nodos y verificación de integridad. El taller combina profundidad teórica con implementación práctica progresiva, cerrando cada semana con un entregable evaluable mediante una rúbrica específica.

Índice

1. Introducción y marco conceptual general **3**
1.1. ¿Qué NO es este taller? . . . 3
1.2. Problema fundamental que resuelve una blockchain: el doble gasto . . . 3

2. Fundamentos teóricos en profundidad **4**
2.1. Funciones hash criptográficas . . . 4
2.1.1. Ejercicio conceptual (no entregable, pero debe resolverse en clase) . . . 4
2.2. Estructura de un bloque y encadenamiento . . . 4
2.2.1. El bloque génesis . . . 5
2.3. Proof of Work (PoW): consenso mediante costo computacional . . . 5
2.3.1. Mecánica . . . 5
2.3.2. ¿Por qué esto asegura el sistema? . . . 5
2.3.3. Ajuste de dificultad . . . 6
2.4. Criptografía asimétrica y firmas digitales . . . 6
2.4.1. ¿Cómo prueban la propiedad de los fondos sin un banco? . . . 6
2.4.2. La "wallet"no guarda dinero . . . 6
2.5. El problema del doble gasto y el mempool . . . 6
2.6. Consenso y descentralización: más allá de un solo nodo . . . 7
2.6.1. Regla de la cadena más larga . . . 7
2.6.2. Proof of Work vs. Proof of Stake (contexto comparativo) . . . 7
2.7. El "trilema"de la blockchain . . . 7

3. Estructura general del taller: 3 semanas **7**

4. Semana 1: Fundamentos – Hash, Bloques y Cadena **9**
4.1. Contenido teórico a cubrir . . . 9
4.2. Actividades prácticas . . . 9

4.3. Rúbrica – Semana 1 (25 % de la nota final) 10

5. Semana 2: Criptografía, Wallets, Proof of Work y Transacciones 11
5.1. Contenido teórico a cubrir 11
5.2. Actividades prácticas 11
5.3. Rúbrica – Semana 2 (40 % de la nota final) 12

6. Semana 3: Red de Nodos, Consenso, API REST y Seguridad 14
6.1. Contenido teórico a cubrir 14
6.2. Actividades prácticas 14
6.3. Pruebas de seguridad mínimas exigidas 15
6.4. Rúbrica – Semana 3 / Entrega final (35 % de la nota final) 15

7. Rúbrica consolidada y nota final 17
7.1. Errores comunes a advertir a los estudiantes 17

8. Anexo: gamificación del taller – «Crea tu propia criptomoneda» 18
8.1. Nombre de la actividad 18
8.2. Ficha de identidad del equipo 18
8.3. Sistema de retos por checkpoint 19
8.4. Reto final entre equipos: "Hackea la blockchain" 19
8.5. Extensión interdisciplinar (opcional): detección de anomalías con Machine Learning . 20

9. Extensiones opcionales para equipos avanzados 21

1 Introducción y marco conceptual general

Una criptomoneda no es, en el fondo, "dinero digital.[en] el sentido simple de un número en una base de datos. Es la combinación de cuatro ingredientes que, por separado, ya existían mucho antes de Bitcoin (2008), pero que nunca se habían ensamblado de esta forma:

1. **Una estructura de datos verificable e inmutable** (la blockchain), construida sobre funciones hash.
2. **Criptografía asimétrica** para probar la propiedad de los fondos sin revelar una identidad ni depender de un tercero.
3. **Un mecanismo de consenso** que permite que participantes que no se conocen ni confían entre sí se pongan de acuerdo sobre un único historial de transacciones válido.
4. **Incentivos económicos** que alinean el interés individual de cada participante con la seguridad del sistema en conjunto.

→ *Notarán que los primeros tres son estrictamente técnicos y el cuarto es económico/ de teoría de juegos – ambos importan.*

El objetivo de este taller es que cada estudiante construya, con sus propias manos, cada uno de estos cuatro componentes, entendiendo *por qué* existen y no solo *cómo* se implementan. Un error común en talleres de blockchain es copiar código de un tutorial sin entender qué problema resuelve cada pieza; este taller está diseñado explícitamente para evitar eso, exigiendo justificación conceptual en cada entrega.

1.1 ¿Qué NO es este taller?

- No es la creación de una criptomoneda real"que cotice en un exchange – eso implica auditorías de seguridad, cumplimiento regulatorio y infraestructura de red que exceden el alcance académico.
- No requiere conocimientos previos de blockchain, pero sí bases sólidas de estructuras de datos, funciones y POO.
- No es un ejercicio de copiar y pegar: cada entregable exige una justificación técnica escrita, no solo código que funcione.

1.2 Problema fundamental que resuelve una blockchain: el doble gasto

Antes de Bitcoin, el gran problema no resuelto del dinero digital era el **doble gasto** (*double-spending*): si el dinero es solo información (bits), ¿qué impide que alguien copie esa información y la gaste dos veces? En el mundo físico, un billete no se puede çopiar"fácilmente y usar dos veces; en el mundo digital, copiar información es trivial.

La solución tradicional era un **tercero de confianza** (un banco) que llevara un libro contable centralizado. La innovación de Bitcoin fue demostrar que ese libro contable puede mantenerse de forma **distribuida**, sin ningún tercero central, siempre que se resuelva un problema more profundo de ciencias de la computación conocido como el **Problema de los Generales Bizantinos**: ¿cómo logran ponerse de acuerdo varios nodos que no confían entre sí (y donde algunos podrían incluso ser maliciosos), sobre un único estado compartido de la verdad?

Nota importante

Este es el hilo conductor de todo el taller: cada pieza técnica que van a construir (hash, PoW, firmas, consenso) es la respuesta a una pregunta muy concreta de "¿cómo evito que me engañen sin depender de un árbitro central?". Si en algún momento no ven para qué sirve algo, vuelvan a esta pregunta.

2 Fundamentos teóricos en profundidad

2.1 Funciones hash criptográficas

Una función hash criptográfica H toma una entrada de tamaño arbitrario y produce una salida de tamaño fijo (el *digest*). Para SHA-256, la salida siempre son 256 bits (64 caracteres hexadecimales), sin importar si la entrada es una letra o un archivo de 10 GB.

Propiedades que la hacen útil para blockchain:

- **Determinismo:** la misma entrada siempre produce la misma salida.
- **Efecto avalancha:** cambiar un solo bit de la entrada produce una salida completamente distinta e impredecible. Esto es lo que permite detectar manipulación: si alguien altera un bloque, su hash cambia por completo.
- **Resistencia a preimagen:** dado un hash h, es computacionalmente inviable encontrar una entrada x tal que $H(x) = h$.
- **Resistencia a colisiones:** es computacionalmente inviable encontrar dos entradas distintas $x_1 \neq x_2$ tales que $H(x_1) = H(x_2)$.
- **Velocidad de cómputo directo vs. costo de romperla":** calcular $H(x)$ es rápido, pero no existe atajo conocido para invertir la función – la única forma de "buscarüna entrada que produzca cierto resultado es probar por fuerza bruta. Esta asimetría es exactamente lo que se explota en el Proof of Work (Sección 2.3).

2.1.1 Ejercicio conceptual (no entregable, pero debe resolverse en clase)

Calculen el SHA-256 de la palabra `ÜMB"` y luego de `ÜMB."` (con un punto al final) y comparen las salidas. Esto ilustra el efecto avalancha de forma tangible antes de programar nada.

2.2 Estructura de un bloque y encadenamiento

Cada bloque de la cadena contiene, como mínimo:

Campo	Función
`index`	Posición del bloque dentro de la cadena
`timestamp`	Momento de creación del bloque
`transactions`	Lista de transacciones incluidas en ese bloque
`previous_hash`	Hash del bloque inmediatamente anterior
`nonce`	Número usado una sola vez, ajustado durante la minería
`hash`	Hash propio del bloque, calculado sobre todos los campos anteriores

El campo `previous_hash` es lo que convierte una simple lista de bloques en una **cadena verificable**: como el hash de cada bloque depende (entre otras cosas) del hash del bloque anterior, modificar un bloque antiguo obligaría a recalcular **todos** los hashes de los bloques posteriores para que la cadena siga siendo válida. En un sistema con muchos nodos y Proof of Work, esto se vuelve computacionalmente inviable – de ahí la inmutabilidad práctica (no absoluta) de una blockchain.

2.2.1 El bloque génesis

El primer bloque de toda cadena no tiene un bloque anterior real, así que su `previous_hash` se define arbitrariamente (comúnmente como una cadena de ceros). Este bloque debe crearse de forma especial en el código, fuera del flujo normal de minería.

2.3 Proof of Work (PoW): consenso mediante costo computacional

El Proof of Work resuelve el problema de "¿quién tiene derecho a proponer el siguiente bloque?"sin necesidad de una autoridad central, imponiendo un **costo computacional** a esa propuesta.

2.3.1 Mecánica

Se define una **dificultad** d: el hash del bloque debe cumplir una condición arbitraria pero verificable, típicamente "debe comenzar con d ceros". Como el hash es impredecible (efecto avalancha), la única forma de encontrar un `nonce` que cumpla la condición es **probar valores por fuerza bruta**:

$$\text{buscar } n \text{ tal que } H(\text{index} \,\|\, \text{timestamp} \,\|\, \text{transactions} \,\|\, \text{previous_hash} \,\|\, n) \text{ comienza con } \underbrace{00\ldots0}_{d}$$

2.3.2 ¿Por qué esto asegura el sistema?

- **Encontrar** el nonce correcto es lento (proporcional a 2^{4d} intentos esperados, en hexadecimal).
- **Verificar** que un nonce es correcto es instantáneo (un solo cálculo de hash).
- Esta asimetría significa que atacar la red (reescribir el historial) requiere rehacer todo el trabajo computacional de los bloques afectados *más rápido* que el resto de la red honesta combinada – de ahí el famoso .ªtaque del 51 %": solo es viable si un atacante controla más de la mitad del poder de cómputo total de la red.

2.3.3 Ajuste de dificultad

En redes reales como Bitcoin, la dificultad se recalibra cada cierto número de bloques para mantener un tiempo promedio de minado constante (10 minutos por bloque), sin importar cuánto poder de cómputo total tenga la red en ese momento. En este taller no es obligatorio implementar el ajuste dinámico (queda como extensión opcional en la Semana 3), pero sí deben poder explicar por qué es necesario: si la red crece, más nodos minando más rápido sin ajuste de dificultad producirían bloques cada vez más rápido, desestabilizando la emisión de la moneda.

2.4 Criptografía asimétrica y firmas digitales

A diferencia de la criptografía simétrica (una sola llave para cifrar y descifrar), la criptografía asimétrica usa un **par de llaves matemáticamente relacionadas**: una **privada** (secreta, nunca se comparte) y una **pública** (se puede distribuir libremente).

2.4.1 ¿Cómo prueban la propiedad de los fondos sin un banco?

Cuando un usuario quiere enviar fondos, **firma** la transacción con su llave privada. Cualquier nodo de la red puede **verificar** esa firma usando la llave pública correspondiente, sin necesidad de conocer la llave privada. Esto prueba matemáticamente que quien envió la transacción posee la llave privada asociada a esa dirección, sin revelarla.

	Firmar	**Verificar**
Quién lo hace	El dueño de los fondos (emisor)	Cualquier nodo de la red
Llave usada	Privada	Pública
Resultado	Una firma digital adjunta a la transacción	Verdadero/Falso

2.4.2 La "wallet"no guarda dinero

Un malentendido frecuente: una wallet (billetera) no çontiene"fondos como si fuera un cofre. Una wallet es, en esencia, **un par de llaves**. La "dirección"pública deriva de la llave pública (usualmente aplicándole otra función hash). El saldo asociado a una dirección no se almacena en ningún lado de forma directa: se **calcula** recorriendo el historial completo de transacciones de la cadena.

2.5 El problema del doble gasto y el mempool

Antes de que una transacción quede confirmada dentro de un bloque minado, vive en el **mempool** (memory pool): una lista de transacciones pendientes que cada nodo mantiene localmente. El doble gasto se previene en dos capas:

1. **A nivel de nodo individual:** antes de aceptar una transacción en el mempool, el nodo verifica que el emisor realmente tenga saldo suficiente (calculado recorriendo la cadena confirmada).
2. **A nivel de red:** si dos transacciones que gastan el mismo fondo llegan a distintos nodos casi al mismo tiempo, solo una de ellas terminará incluida en el bloque que finalmente gane el consenso; la otra queda inválida.

2.6 Consenso y descentralización: más allá de un solo nodo

Con un único nodo, no hay nada que çonsensuar- ese nodo decide todo. La descentralización real aparece cuando **múltiples nodos independientes** mantienen copias de la cadena y deben ponerse de acuerdo sobre cuál es la versión válida.

2.6.1 Regla de la cadena más larga

La regla de consenso más simple (usada conceptualmente por Bitcoin) es: **ante dos cadenas válidas en conflicto, gana la más larga** (o, más precisamente, la que representa mayor trabajo computacional acumulado). Esto incentiva a los nodos honestos a construir siempre sobre la última cadena válida conocida, en lugar de bifurcar el sistema.

2.6.2 Proof of Work vs. Proof of Stake (contexto comparativo)

	Proof of Work (PoW)	**Proof of Stake (PoS)**
Derecho a proponer bloque	Se "ganaresolviendo un problema computacional	Se asigna probabilísticamente según cuánta moneda se tiene .apostada"(*staked*)
Costo de atacar	Poder de cómputo (hardware + energía)	Capital económico bloqueado en la red
Consumo energético	Alto (Bitcoin)	Significativamente menor (Ethereum desde 2022)
Ejemplo real	Bitcoin	Ethereum (post-*The Merge*)

Este taller implementa PoW por ser conceptualmente más directo de programar y explicar, pero cada equipo debe poder argumentar, al menos en un párrafo, las ventajas y desventajas de ambos enfoques (se evalúa en la rúbrica de la Semana 3).

2.7 El "trilema"de la blockchain

Se suele afirmar que una blockchain solo puede optimizar completamente dos de estas tres propiedades a la vez: **descentralización**, **seguridad** y **escalabilidad**. Este marco conceptual, aunque simplificado, es útil para que los estudiantes justifiquen las decisiones de diseño de su propia implementación (por ejemplo: .elegimos una dificultad de PoW baja para priorizar velocidad de pruebas en clase, sacrificando seguridad realista").

3 Estructura general del taller: 3 semanas

El taller se organiza en tres semanas de trabajo progresivo. Cada semana profundiza sobre la anterior: no es posible completar la Semana 2 sin una cadena funcional de la Semana 1, ni la Semana 3 sin wallets y transacciones firmadas de la Semana 2.

Semana	Foco central	Peso en la nota
1	Fundamentos: hash, bloque, cadena, validación de integridad	25 %
2	Criptografía: wallets, firmas digitales, PoW, transacciones y balances	40 %
3	Sistema completo: red de nodos, consenso, API REST, seguridad y sustentación	35 %

Nota importante

El nombre de la criptomoneda es libre – cada equipo la bautiza como quiera (símbolo, nombre, logo opcional). Esto no afecta la evaluación técnica, pero sí debe mantenerse consistente en todo el código y la documentación entregada.

4 Semana 1: Fundamentos – Hash, Bloques y Cadena

Objetivos de la semana

- Comprender y poder explicar las propiedades de las funciones hash criptográficas.
- Diseñar e implementar la estructura de un bloque.
- Implementar el bloque génesis y el encadenamiento mediante `previous_hash`.
- Implementar una función de validación de integridad de toda la cadena.

4.1 Contenido teórico a cubrir

- Funciones hash: determinismo, efecto avalancha, resistencia a colisiones (Sección 2.1).
- Estructuras de datos enlazadas aplicadas a blockchain (Sección 2.2).
- El problema del doble gasto y por qué la inmutabilidad importa (Sección 1.2).

4.2 Actividades prácticas

1.1 Implementar la clase `Block` con los campos descritos en la Sección 2.2 y un método `calculate_hash()` usando SHA-256 (librería `hashlib`).

1.2 Implementar la clase `Blockchain`, incluyendo la creación automática del bloque génesis.

1.3 Implementar el método `add_block()` que añada un nuevo bloque enlazado correctamente al último bloque existente.

1.4 Implementar `is_chain_valid()`, que recorra toda la cadena verificando que:

- El `hash` almacenado de cada bloque coincide con el hash recalculado sobre sus campos actuales.
- El `previous_hash` de cada bloque coincide exactamente con el `hash` real del bloque anterior.

1.5 **Prueba de manipulación:** alterar manualmente un campo de un bloque intermedio ya creado (por ejemplo, cambiar el monto de una transacción) y demostrar que `is_chain_valid()` detecta la inconsistencia.

Listing 1: Esqueleto de referencia para la clase Block

```
import hashlib
import json
import time

class Block:
    def __init__(self, index, transactions, previous_hash, nonce=0):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.calculate_hash()

```

```
    def calculate_hash(self):
        contenido = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce
        }, sort_keys=True).encode()
        return hashlib.sha256(contenido).hexdigest()
```

Entregable – Semana 1 (individual o por equipo, según defina el docente)

- Repositorio de código (GitHub) con las clases `Block` y `Blockchain` funcionales.
- Script o notebook de demostración que cree al menos 5 bloques encadenados.
- Evidencia (captura o log) de la prueba de manipulación: cadena válida antes de alterar un bloque, e inválida después.
- Documento corto (máx. 1 página) respondiendo: ¿por qué el efecto avalancha del hash es indispensable para detectar manipulación? ¿Qué pasaría si se usara una función hash sin esa propiedad?
- **Fecha de entrega:** final de la Semana 1 (definida por el docente en el cronograma del curso).

4.3 Rúbrica – Semana 1 (25 % de la nota final)

Criterio	Peso	Descripción de niveles de logro
Estructura del bloque	20 %	**Excelente:** todos los campos requeridos, correctamente tipados y serializados. **Aceptable:** faltan campos menores o serialización inconsistente. **Insuficiente:** estructura incompleta o hash mal calculado.
Encadenamiento y bloque génesis	20 %	**Excelente:** génesis correcto, cada bloque referencia con exactitud el hash real del anterior. **Aceptable:** encadenamiento funcional con errores menores en casos borde. **Insuficiente:** bloques no verdaderamente encadenados.
Validación de integridad (`is_chain_valid`)	30 %	**Excelente:** detecta correctamente cualquier alteración en cualquier bloque, incluyendo alteraciones al `previous_hash`. **Aceptable:** detecta la mayoría de casos pero falla en alguno borde. **Insuficiente:** no detecta manipulación o arroja falsos positivos.
Evidencia de prueba de manipulación	15 %	Evidencia clara, reproducible, con capturas o logs del antes/después.
Justificación conceptual escrita	15 %	Explicación correcta, en palabras propias, del efecto avalancha y su rol en la detección de manipulación.

5 Semana 2: Criptografía, Wallets, Proof of Work y Transacciones

Objetivos de la semana

- Comprender e implementar la generación de pares de llaves y su uso para firmar y verificar transacciones.
- Implementar el Proof of Work (minería) sobre la estructura construida en la Semana 1.
- Implementar el mempool de transacciones pendientes y el cálculo de balances.
- Implementar la recompensa de minería.

5.1 Contenido teórico a cubrir

- Criptografía asimétrica: llave pública vs. privada, firma vs. verificación (Sección 2.4).
- Proof of Work: mecánica, asimetría de costo búsqueda/verificación, ataque del 51 % (Sección 2.3).
- El mempool y la prevención de doble gasto (Sección 2.5).

5.2 Actividades prácticas

2.1 Implementar `create_wallet()` generando un par de llaves (se recomienda la librería `ecdsa` sobre la curva secp256k1, la misma que usa Bitcoin, con fines didácticos).

2.2 Derivar una "dirección"pública a partir de la llave pública (por ejemplo, aplicando SHA-256 sobre la llave pública).

2.3 Implementar `sign_transaction()`: dada una transacción (`de`, `para`, `monto`) y la llave privada del emisor, producir una firma digital.

2.4 Implementar `verify_transaction()`: dada la transacción, la firma y la llave pública del emisor, verificar que la firma es válida.

2.5 Implementar el `mempool`: lista de transacciones válidas pendientes de confirmación.

2.6 Modificar `mine_block()` (introducido conceptualmente en la Sección 2.3) para que: tome las transacciones del mempool, busque un `nonce` válido según la dificultad definida, y añada una transacción de **recompensa de minería** a favor de quien minó el bloque.

2.7 Implementar `get_balance(direccion)`, recorriendo toda la cadena confirmada.

2.8 **Prueba obligatoria:** intentar aceptar una transacción con firma inválida o falsificada, y demostrar que el sistema la rechaza *antes* de que entre al mempool.

Listing 2: Esqueleto de referencia para firma y verificación

```
from ecdsa import SigningKey, VerifyingKey, SECP256k1
import hashlib

def create_wallet():
    private_key = SigningKey.generate(curve=SECP256k1)
    public_key = private_key.get_verifying_key()
    return private_key, public_key
```

```python

def sign_transaction(private_key, transaction_data: str):
    return private_key.sign(transaction_data.encode())

def verify_transaction(public_key, transaction_data: str, signature) -> bool:
    try:
        return public_key.verify(signature, transaction_data.encode())
    except Exception:
        return False
```

Nota importante

La dificultad del Proof of Work debe calibrarse para fines didácticos: 4 ceros iniciales en SHA-256 suele tardar segundos; 6 o más puede tardar minutos u horas en un computador portátil normal. Se recomienda exponer la dificultad como un parámetro configurable para poder demostrarla en clase con distintos valores.

Entregable – Semana 2

- Código funcional de wallets, firma/verificación de transacciones, mempool, minería con PoW y cálculo de balances.
- Demostración (script o notebook) de un flujo completo: crear 2 wallets → transferir fondos entre ellas → minar el bloque → consultar balances actualizados.
- Evidencia de la prueba de rechazo de firma inválida.
- Tabla comparativa (puede ser parte del mismo documento) del tiempo de minado con al menos 3 niveles de dificultad distintos, con una breve interpretación de los resultados.
- Párrafo comparativo Proof of Work vs. Proof of Stake (ver Sección 2.6.2), con al menos una ventaja y una desventaja de cada uno.
- **Fecha de entrega:** final de la Semana 2.

5.3 Rúbrica – Semana 2 (40 % de la nota final)

Criterio	Peso	Descripción de niveles de logro
Wallets y generación de llaves	15 %	Generación correcta de pares de llaves y derivación consistente de direcciones.
Firma y verificación de transacciones	25 %	**Excelente:** firma y verificación correctas, rechazo comprobado de firmas inválidas o falsificadas. **Aceptable:** funciona en el caso feliz pero no valida casos maliciosos. **Insuficiente:** acepta transacciones sin verificar la firma.
Proof of Work funcional	20 %	Minería correcta según dificultad configurable, con evidencia de tiempos variables según dificultad.
Mempool, balances y recompensa de minería	20 %	Flujo completo transacción → mempool → bloque minado → balance actualizado, incluyendo recompensa correcta al minero.

Criterio	Peso	Descripción de niveles de logro
Análisis comparativo PoW vs. PoS	10 %	Argumentación correcta y propia (no copiada literalmente de una fuente), con al menos una ventaja/desventaja de cada modelo.
Calidad de la evidencia entregada	10 %	Tabla de tiempos de minado por dificultad, con interpretación breve y coherente de la tendencia observada.

6 Semana 3: Red de Nodos, Consenso, API REST y Seguridad

Objetivos de la semana

- Exponer el nodo blockchain mediante una API REST.
- (Opcional/avanzado) Levantar una red de al menos dos nodos con sincronización por consenso.
- Diseñar y ejecutar pruebas de seguridad sobre el sistema completo.
- Documentar y sustentar el proyecto de forma integral.

6.1 Contenido teórico a cubrir

- Consenso distribuido y la regla de la cadena más larga (Sección 2.6.1).
- El Problema de los Generales Bizantinos, revisitado con la implementación ya construida (Sección 1.2).
- El "trilema"de la blockchain: descentralización, seguridad, escalabilidad (Sección 2.7).

6.2 Actividades prácticas

3.1 Implementar un servidor REST (Flask o FastAPI) que exponga como mínimo:

- `POST /transacciones/nueva` – registrar una transacción firmada en el mempool.
- `GET /cadena` – devolver la blockchain completa.
- `GET /minar` – forzar el minado de un nuevo bloque con las transacciones pendientes.
- `GET /balance/<direccion>` – consultar el balance de una dirección.

3.2 **(Avanzado, obligatorio solo para nota máxima)** Levantar 2 o 3 instancias del nodo en puertos distintos, implementando:

- `POST /nodos/registrar` – que un nodo conozca la existencia de otro.
- `GET /nodos/resolver` – aplicar la regla de la cadena más larga válida entre todos los nodos conocidos.

3.3 Diseñar y ejecutar, como mínimo, tres pruebas de seguridad (ver rúbrica) usando `pytest` u otro framework de pruebas.

3.4 Preparar documentación final: `README.md` con instrucciones de instalación y ejecución, diagrama de arquitectura del sistema (bloques, wallets, nodos, API) y manual breve de uso de los endpoints.

3.5 Preparar sustentación en vivo (ver criterios en la rúbrica).

Nota importante

El punto 3.2 (red de nodos) es lo que diferencia un proyecto çompleto"de uno "sobresaliente". Los equipos que no lo implementen pueden alcanzar máximo el nivel .ªceptable.en el criterio de descentralización de la rúbrica, pero sí deben poder explicar conceptualmente cómo lo harían.

Entregable – Semana 3 (entrega final del taller)

- Repositorio de GitHub completo y organizado, con `README.md` claro.
- Nodo(s) ejecutable(s) localmente, probado(s) con Postman/Insomnia/cURL (incluir colección o comandos de ejemplo).
- Suite de pruebas de seguridad (mínimo 3 casos, ver rúbrica) con resultados documentados.
- Diagrama de arquitectura del sistema completo.
- Sustentación en vivo ante el curso (10-15 min por equipo): crear una wallet, realizar una transacción, minar un bloque y mostrar el balance actualizado en tiempo real; si se implementó la red de nodos, demostrar también la sincronización entre ellos.
- **Fecha de entrega:** final de la Semana 3, coincide con la sustentación.

6.3 Pruebas de seguridad mínimas exigidas

(a) **Rechazo de firma inválida:** enviar una transacción con una firma alterada o generada con una llave privada distinta a la del supuesto emisor, y verificar que el sistema la rechaza.

(b) **Detección de manipulación de un bloque confirmado:** modificar directamente un campo de un bloque ya minado en la estructura en memoria (o en el almacenamiento persistente, si lo hay) y verificar que `is_chain_valid()` lo detecta.

(c) **Rechazo de doble gasto:** intentar registrar dos transacciones que gasten el mismo saldo antes de que ninguna se confirme, y verificar que el sistema no permite que ambas se minen (o que la segunda es rechazada por saldo insuficiente calculado sobre el estado real).

6.4 Rúbrica – Semana 3 / Entrega final (35 % de la nota final)

Criterio	Peso	Descripción de niveles de logro
API REST funcional	20 %	Todos los endpoints mínimos implementados y probados, con manejo correcto de errores (por ejemplo, transacción mal formada).
Red de nodos y consenso (descentralización)	20 %	**Sobresaliente:** 2+ nodos sincronizados con regla de cadena más larga demostrada en vivo. **Aceptable:** un solo nodo funcional, pero con explicación conceptual clara de cómo se extendería a red. **Insuficiente:** no se puede explicar el mecanismo de consenso.
Pruebas de seguridad (a, b, c)	25 %	Las tres pruebas mínimas implementadas, ejecutadas y documentadas con resultados reales (no solo descritas en teoría).
Documentación (`README`, diagrama de arquitectura)	15 %	Documentación suficiente para que un tercero clone el repositorio y ejecute el sistema sin ayuda del equipo.

Criterio	**Peso**	**Descripción de niveles de logro**
Sustentación en vivo	20 %	Dominio conceptual del equipo (todos los integrantes pueden responder preguntas), demostración fluida del flujo completo, manejo adecuado del tiempo asignado.

7 Rúbrica consolidada y nota final

Semana	Entregable	Peso
Semana 1	Cadena básica con validación de integridad	25 %
Semana 2	Wallets, firmas, PoW, transacciones y balances	40 %
Semana 3	Sistema completo, seguridad y sustentación	35 %
Total		**100 %**

Nota importante

El docente puede ajustar los pesos relativos según la duración real de las sesiones de clase, pero se recomienda mantener a la Semana 2 como la de mayor peso, dado que concentra la mayor densidad conceptual (criptografía asimétrica + consenso computacional) y suele ser el punto donde más dificultades surgen.

7.1 Errores comunes a advertir a los estudiantes

- Confundir la dirección pública de una wallet con la llave privada – la privada **nunca** se transmite ni se expone en logs, respuestas de API o repositorios públicos.
- Omitir algún campo en el cálculo del hash del bloque al validar la cadena, lo que produce falsos positivos o negativos en `is_chain_valid()`.
- Calibrar una dificultad de PoW demasiado alta (8+ ceros en SHA-256), lo que puede hacer que la minería tarde horas en un computador portátil estándar.
- Aceptar transacciones en el mempool sin verificar la firma *antes* de aceptarlas, dejando abierta la puerta a transacciones falsificadas.
- Calcular el balance sumando/restando solo transacciones del mempool en lugar de recorrer la cadena **confirmada**, lo que puede permitir gastar fondos no confirmados.

8 Anexo: gamificación del taller – «Crea tu propia criptomoneda»

Todo el contenido técnico de las Secciones 4 a 6 puede envolverse en una dinámica de juego/competencia sin alterar los objetivos de aprendizaje ni las rúbricas ya definidas. Esta sección es **opcional** para el docente: puede usarse como motivador transversal a las tres semanas, o reservarse como actividad de cierre. Los retos aquí descritos **no reemplazan** los entregables formales de las Secciones 4–6; los complementan como puntos adicionales de participación o como guía de ritmo dentro de cada sesión de laboratorio.

8.1 Nombre de la actividad

«Crea tu propia criptomoneda»

El reto general que se presenta a cada grupo desde el primer día es:

Misión del equipo

Construir una criptomoneda funcional en Python capaz de registrar transacciones, crear bloques, calcular hashes y realizar minería – y defenderla de los intentos de manipulación de los demás equipos.

8.2 Ficha de identidad del equipo

Antes de programar, cada grupo debe llenar su propia "ficha de identidad", que se entrega junto con el resto de la documentación de la Semana 1. Esto ya se menciona como libertad de nombre en la Sección 3, pero aquí se formaliza con una plantilla:

Identidad	
Nombre de la criptomoneda	_____
Símbolo (3–5 letras)	_____
Logo (opcional)	_____
Economía	
Cantidad inicial en circulación	_____
Recompensa por bloque minado	_____
Parámetros de la blockchain	
Dificultad de minería (# de ceros)	_____
Algoritmo de hash	_____

Nombres de ejemplo para inspirar a los grupos que no tengan idea: *TecnoCoin, IACoin, UniCoin, JaguarCoin, SoftwareCoin, CampusCoin, UMBCoin.*

8.3 Sistema de retos por checkpoint

Los retos siguientes son puntos de control (çheckpoints") alineados con las actividades ya descritas en las Secciones 4 a 6. Se sugiere llevarlos como una lista visible en el aula (físico o tablero digital) donde cada equipo marca su avance – esto añade el componente competitivo/lúdico sin cambiar el contenido técnico exigido.

Reto 1– Génesis

Crear el bloque génesis de la cadena. *(Corresponde a la actividad 1.2, Sección 4.)*

Reto 2– Primera transacción

Registrar la primera transacción entre dos usuarios inventados por el equipo. *(Actividad 1.1–1.2.)*

Reto 3– El hash cambia todo

Generar el hash de una transacción, modificar un solo carácter y demostrar en clase cómo cambia por completo el resultado (efecto avalancha, Sección 2.1). Este es el .experimento del carácterz funciona muy bien como demostración en vivo ante el resto del curso.

Reto 4– Tres bloques encadenados

Construir y encadenar al menos tres bloques, cada uno referenciando correctamente el hash del anterior.

Reto 5– Primera minería

Minar un bloque con dificultad de 2 ceros y luego repetir con 3, 4 y 5 ceros, registrando intentos y tiempo en una tabla (ver plantilla de la Sección 5.2, "Prueba de manipulaciónz el entregable de tiempos de minado de la Semana 2).

Reto 6– Detectar el fraude

Detectar una transacción manipulada dentro de la propia cadena del equipo, ejecutando `is_chain_valid()`.

Reto 7– Cadena completa validada

Validar toda la blockchain del equipo de principio a fin, sin errores, ante el docente.

Reto 8– Contraataque

Intentar vulnerar la blockchain de otro equipo (ver dinámica completa en la Sección 8.4).

8.4 Reto final entre equipos: “Hackea la blockchain”

Esta es la dinámica de cierre recomendada para la Semana 3, ejecutable como actividad presencial de 30–45 minutos antes o durante la sustentación final. Convierte la clase, por un momento, en un

pequeño laboratorio de ataque/defensa (similar en espíritu a un ejercicio de *red team / blue team* de ciberseguridad).

1. Cada equipo entrega su blockchain (código + estado de la cadena, por ejemplo exportada a JSON o accesible vía el endpoint `GET /cadena` de la Sección 6) a otro equipo, asignado por el docente.
2. El equipo receptor tiene un tiempo fijo (por ejemplo, 15 minutos) para **modificar una transacción dentro de un bloque ya confirmado, sin que la manipulación sea evidente a simple vista** – es decir, sin romper la estructura JSON ni dejar rastros obvios.
3. Vencido el tiempo, la blockchain modificada se devuelve al equipo original (o se pone a disposición de un tercer equipo .ªuditor").
4. El equipo auditor ejecuta `is_chain_valid()` sobre la cadena recibida y debe reportar: **(a)** si la cadena es válida o no, y **(b)** en qué bloque específico se detecta la inconsistencia (comparando `hash` almacenado vs. recalculado, y `previous_hash` vs. hash real del bloque anterior).
5. **Puntuación sugerida:** puntos de bonificación tanto para el equipo atacante si logra que la manipulación no sea detectada de inmediato, como para el equipo auditor si identifica correctamente el bloque exacto donde ocurrió la alteración. Esto incentiva rigor en ambos roles.

Nota importante

Esta dinámica solo funciona – y solo tiene valor pedagógico – si `is_chain_valid()` está bien implementada (Semana 1) y si la estructura del hash cubre realmente todos los campos relevantes del bloque. Si algún equipo logra manipular la cadena sin que la validación lo detecte, eso en sí mismo es un hallazgo valioso: revela un defecto real en su función de validación, y debe discutirse abiertamente en la sustentación en lugar de penalizarse como una falla, ya que ilustra directamente el objetivo de la Sección 8.4 (Pruebas de seguridad).

8.5 Extensión interdisciplinar (opcional): detección de anomalías con Machine Learning

Para cursos con componente de Inteligencia Artificial, el proyecto puede extenderse una vez completada la Semana 3: con el historial de transacciones ya generado (propio y el de los intentos de ataque de la Sección 8.4), los estudiantes pueden entrenar un modelo simple de detección de anomalías (por ejemplo, aislamiento de outliers sobre montos y frecuencia de transacciones por dirección) que señale transacciones sospechosas antes de que se incluyan en un bloque. Esta extensión es independiente de la nota del taller de blockchain, pero conecta directamente con cursos de Ciberseguridad e IA y puede ofrecerse como proyecto de profundización o de crédito adicional.

9 Extensiones opcionales para equipos avanzados

- Interfaz web simple (React o HTML/JS plano) para crear wallets y transacciones sin depender de Postman.
- Ajuste dinámico de dificultad de PoW según el tiempo real de minado de los últimos n bloques (ver Sección 2.3.3).
- Un modelo simplificado de "smart contract": una función que se ejecute automáticamente cuando se cumpla una condición definida en una transacción (por ejemplo, un pago condicionado a una fecha).
- Persistencia de la cadena en disco (por ejemplo, en SQLite o archivos JSON) en lugar de mantenerla solo en memoria.

Taller elaborado por Javier Ochoa – Universidad Manuela Beltrán (UMB)